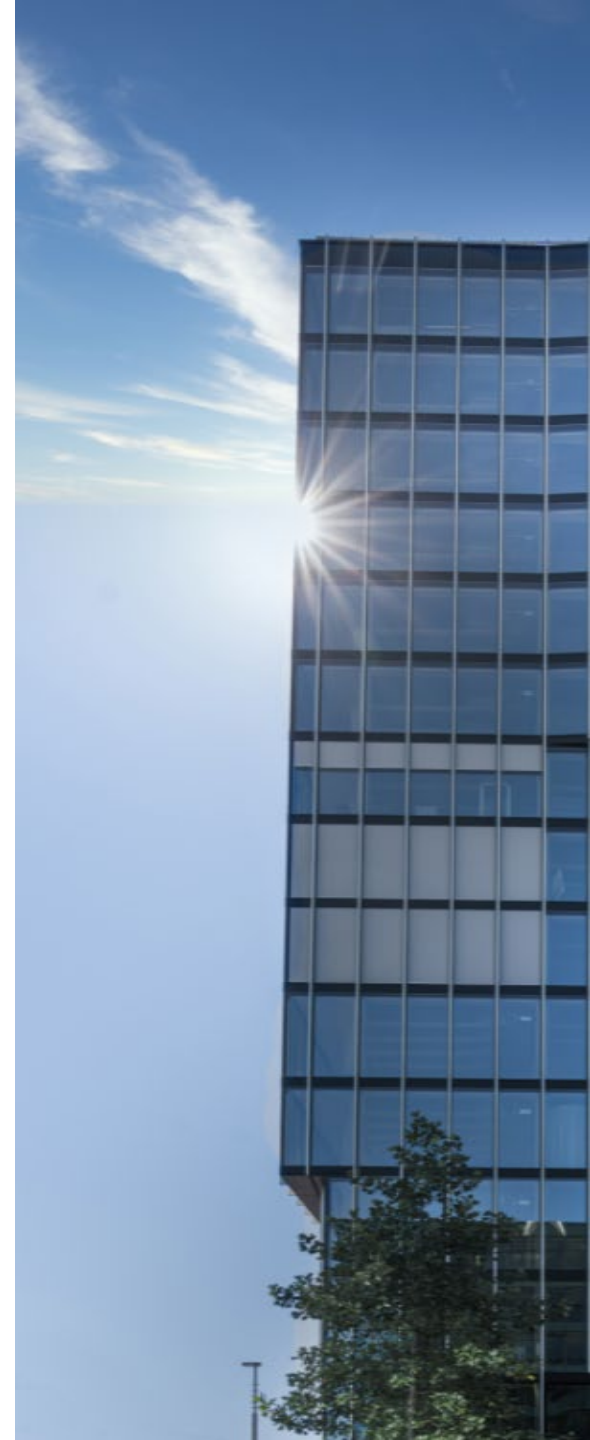


Objektorientierte Programmierung

# Designprinzipien

**Zusammenfassung von zehn konkreten Empfehlungen**

Roland Gisler



# Inhalt

- Zehn grundlegende Designprinzipien!
- Eine Hilfestellung, damit Sie besseres OO-Design entwerfen, und besseren Quellcode schreiben können.
- Zusammenfassung aus den bisherigen Inhalten aber auch den Erfahrungen aus allen Übungen.
- Ein Schritt in Richtung «Software Craftsmanship».

# Lernziele

- Sie kennen zehn fundamentale Designprinzipien.
- Sie können diese Prinzipien in der Programmierung anwenden.
- Sie verstehen den Sinn, Zweck und Nutzen dieser Designprinzipien.

# Zehn wichtige Designprinzipien - Übersicht

1. Single Responsibility Principle (SRP).
2. Don't Repeat Yourself (DRY).
3. Favor Composition over Inheritance (FCoI).
4. Design for inheritance, oder prohibit it.
5. Achten Sie auf hohe Kohäsion und lose Kopplung.
6. Differenzieren Sie Datenkapselung von Information Hiding.
7. Verwenden Sie Schnittstellen zur Entkopplung und Abstraktion.
8. Testen Sie mit Unit Tests nach dem Test First Prinzip.
9. Viele kleine Einheiten sind besser als wenige Grosse.
10. Aussagestarke Namensgebung und formatierter Quellcode.

# **Single Responsibility Principle (SRP)**

# Single Responsibility Principle (SRP)

- Unix-Philosophie:  
«Tu nur ein Ding, genau ein Ding, das aber richtig!»
- Eine Klasse hat möglichst nur **eine** Zuständigkeit.
  - Eine Klasse hat nur **«einen»** Grund zur Änderung.
- Änderungen oder Erweiterungen sollten sich immer auf möglichst wenige Klassen beschränken.
  - Hohe Kohäsion bleibt erhalten.
- ➔ Viele **kleine** Klassen sind besser als wenige grosse Klassen.
  - Konsequenz: ➔ stark verbesserte Testbarkeit!

# **Don't Repeat Yourself (DRY)**

# Don't Repeat Yourself (DRY)

- Duplizierung von Code wenn immer möglich **vermeiden**.
- **Einfache** Duplizierung:
  - Kopierte Codefragmente.
- **Subtile** Duplizierung (Beispiele):
  - Wiederholte, gleichartige Bedingungs-Konstrukte.
  - Gleiche Algorithmen, unterschiedlich implementiert.
- Lösungen:  
(Unter-)Funktionen, Objektorientierung, Design Patterns.
- Vermeidung von Duplizierung erhöht die Produktivität!
  - Wiederverwendung wird verbessert, bzw. dadurch erst möglich.
  - Einfachere Fehlerbehebung.



# **Favor Composition over Inheritance (FCoI)**

## Favor Composition over Inheritance (FCoI)

- Im Zweifelsfall:  
Die Komposition (part-of) ist der Vererbung (is-a) vorzuziehen!
- **Keine** Klassen spezialisieren, welche nicht explizit für Vererbung vorgesehen sind.
  - **Ohne** explizite Empfehlung in der Dokumentation:  
Klasse **nicht** spezialisieren (auch wenn sie nicht **final** sind.)
- **Komposition** ist deutlich flexibler und wartungsfreundlicher.
  - Wrapper- / Delegate- oder Decorator-Pattern (GoF).
  - Komposition ist leicht austauschbar, ohne Einfluss auf die Nutzung / Schnittstelle.

**Design for inheritance, or prohibit it.**

## Design for inheritance, or prohibit it.

- Eine gut spezialisierbare (Basis-)Klasse zu schreiben, ist wirklich anspruchsvoll!
- Für eine Spezialisierung müssen Teile der Implementation in der API dokumentiert werden, was implizit der Datenkapselung widerspricht.
- Wenn möglich **keine überschreibbaren** Methoden aufrufen, auf keinen Fall aber in Konstruktoren oder Methoden mit ähnlichem Charakter (z.B. `clone()`)
  - Gefahr der →pränatalen Kommunikation!
  - Ausschliesslich abstrakte Methoden implementieren lassen.
- Im Zweifelsfall (oder Normalfall!) eine Spezialisierung besser explizit verhindern.
  - Klasse mit **final** markieren (und ggf. privatem Konstruktor).

# **Hohe Kohäsion und lose Kopplung**

# Hohe Kohäsion und lose Kopplung

- **Hohe** Kohäsion:

Man fasst in einer Klasse Attribute und Methoden zusammen, die **wirklich** zusammengehören – nicht mehr, und nicht weniger!

- Kohäsion ist optimal, wenn eine weitere Teilung unsinnig ist.

- **Lose** Kopplung:

So viele und so starke Beziehungen zwischen Klassen wie nötig, so wenig und schwache Beziehungen wie möglich.

- Klassen aus den Java-Libraries kann man (fast) ignorieren.
- Abhängigkeiten zu Thirdparty-Klassen jedoch besonders genau betrachten.

- Die ➔ Datenkapselung und das ➔ Information Hiding unterstützen eine hohe Kohäsion und eine lose Kopplung.

# **Datenkapselung und Information Hiding**

# Datenkapselung und Information Hiding

- Datenkapselung:

Expliziter und bewusster Umgang mit Zugriffsmodifikatoren.

- Attribute möglichst immer **private**.
- Aber bitte nicht einfach alle Methoden **public**.
- Blindes Generieren von Setter-/Getter-Methoden reicht nicht!

- Information Hiding:

Schnittstelle und Implementation möglichst **strikt** trennen. Die Schnittstelle sollte möglichst **keine Rückschlüsse** auf die interne Implementation ermöglichen

- Das sind **echte**, sehr wichtige Design-Entscheide.

- Gute Datenkapselung und Information Hiding fördern direkt eine  
➔ hohe Kohäsion und eine ➔ lose Kopplung.



**Verwenden Sie Schnittstellen**

# Verwenden Sie Schnittstellen

- Schnittstellen sind ein wunderbares Mittel zur Abstraktion und Entkopplung über technische «Rollen» (act-as).
- Schnittstellen sind in Java (meistens) die viel bessere Alternative zu vollabstrakten Klassen.
  - Allenfalls ist auch eine Kombination möglich.
- «Design by interfaces»:
  - Man konzentriert sich immer zuerst auf das **was**, und erst später bei der Implementation auf das **wie**.
  - Fokus stärker auf die Perspektive der Nutzung (statt der Implementation) einer Klasse setzen.
- Erleichtert ganz nebenbei die Umsetzung von ➔**Test First** enorm.
- Lieber zu viele Schnittstellen als zu wenig!

**Testen Sie mit Unit Tests nach dem Test First Prinzip**

# Testen Sie mit Unit-Tests nach dem Test First Prinzip

- Wir testen nicht, um nachträglich Fehler zu finden!
- Sondern wir testen, um fortlaufend die Gewissheit zu erhalten, dass es funktioniert!
- Automatisiertes, selbstvalidiertes Testen ist extrem effizient.
  - JUnit als populärstes Framework, weit verbreitet und integriert.
- Test First – Prinzip / Test Driven Development (TDD):  
**Vor** der Implementation die Testfälle schreiben!
  - Früh die Perspektive des Nutzens einer Klasse einnehmen.
  - Spezial- und Sonderfälle werden intuitiver aufgedeckt ...
  - ... und bei der Implementation ebenso intuitiv berücksichtigt.
- **Kein Projekt und keine Klasse ist zu klein, um nicht mit Unit-Tests getestet zu sein!**

**Viele kleine Einheiten sind besser als wenige Grosse**

# Viele kleine Einheiten sind besser als wenige Grosse

- «Einheiten» steht für: Module, Packages, Klassen, Methoden.
- Viele kleine Methoden und Klassen sind viel besser und schneller verständlich, besser wiederverwendbar und auch einfacher testbar!
- Kleinere Einheiten haben meist auch eine höhere Kohäsion!
  - Eine Verschlechterung wird schneller und einfacher erkannt.
- Bei jeder Ergänzung oder Erweiterung, die man macht **immer hinterfragen**, ob es nicht besser wäre eine bestehende Klasse oder Methode aufzuteilen!
  - Kurzfristig zwar höherer Arbeitsaufwand (z.B. wegen Namens-findung!), der sich später aber **n**-fach durch eine einfachere Weiterentwicklung und höhere Wiederverwendung auszahlt.
- Erkenntnis: Refactoring ist ein **ständiger** Prozess!

# **Aussagekräftige Namen und formatierter Quellcode**

# Aussagekräftige Namen und formatierter Quellcode

- Wählen Sie Namen für Klassen und Methoden **nie** leichtfertig!
  - Denken Sie, es wäre der Name Ihres eigenen Kindes!
- Ein guter Name...
  - hilft, den Sinn und die Absicht schneller zu verstehen.
  - reduziert den Aufwand für die Dokumentation.
  - erhält die Kohäsion (weil man Verletzungen schneller erkennt).
- Gute Namensgebung sorgt für aussagekräftigen und leicht-verständlichen Quellcode, der besser wart- und erweiterbar ist.
- Halten Sie Ihren Quellcode immer sauber (und automatisch) korrekt **formatiert!** (siehe [Q05 IP CodeFormatieren](#))
  - Besonders wichtig bei der Arbeit im Team und mit Versionskontrolle, reduziert die Differenzen (in changesets).



# Zusammenfassung

- Zehn **Prinzipien**. Zehn **Empfehlungen**. Zehn **Hinweise**.
- Weitgehend sprachunabhängig.
- Quellen und Inspiration:
  - **Clean Code** von Robert C. Martin.
  - **Effective Java** von Joshua Bloch.
  - Erfahrung von diversen SW-Entwicklern und Dozenten.
  - Eigene, persönliche Erfahrung.
- ➔ Hätte ich selber diese zehn Prinzipien früher gekannt (und beherzigt), wäre sehr vieles besser geworden!
- Mehr Input zu ➔Clean Code gibt es im Modul **VSK**.

# Literaturhinweise - Empfehlungen

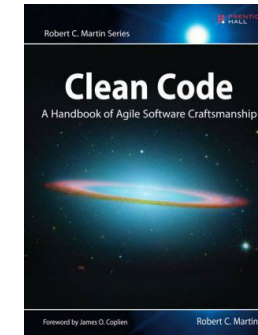
- Robert C. Martin: **Clean Code**

A Handbook of Agile Software Craftsmanship.

Prentice Hall, März 2009

ISBN: 978-0-13-235088-4 (englisch)

ISBN: 978-3-8266-5548-7 (deutsch)

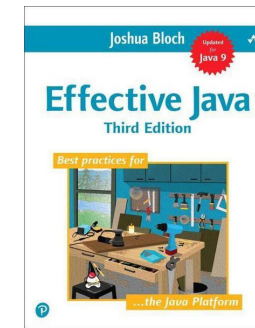


- Joshua Bloch: **Effective Java**

Best practices for the Java Plattform.

Pearson Addison-Wesley, Third Edition, Januar 2018

ISBN: 978-0-13-468599-1





**Fragen?**